Name: Joel Pam Pam ID# 1001428105

Date Submitted: 4/26/2026 Time Submitted 9:20 PM

CSE 3341 – Digital Logic Design II
CSE 5357 – Advanced Digital Logic Design

Spring Semester 2026

Term Project

Half-Precision Floating-Point Adder Subtractor

(100+ Points)

Due by April 28, 2026, 11:59 pm

Submit through Canvas Assignments

Requirements Summary:

This project focuses on doing half precision floating point addition and subtraction based on the IEEE 754 standards. Inputs A and B are entered in as Hex values to represent the 16 bit binary numbers and the output is also displayed as a Hex value on the hex display.

Input Requirements:

1. Input must be entered on the CSE 4 x 4 keyboard
2. Input must be entered in binary IEEE format which can be done by entering in as a HEX value. For example, 3.75 in IEEE binary format is 0100001110000000 which should be entered in as 4380 on the keyboard
3. Operand A should be entered by keying in the number and pressing key 2 on the Hex Board
4. Operand B should be entered by keying in the number and pressing key 3 on the Hex Board
5. Results R will be computed by pressing key 4 on the Hex Board
6. Key 0 on the DE10 can be used to reset

Output Requirements:

1. Operand A will be display on the Hex display in hex according to IEEE 754 half precision format
2. Operand B will be displayed on the Hex display in hex according to IEEE 754 half precision format after pressing key 3
3. Result R will be displayed on the Hex display in hex according to IEEE 754 half precision format after pressing key 4
4. LEDR0 on the DE10 will showcase the Overflow Flag
5. LEDR1 on the DE10 will showcase the Underflow Flag

Complete process to use the floating adder/subtractor on the DE10:

1. Enter operand A in hex based on the IEEE 754 half precision format and press key 2. This will display A on the Hex display
2. Enter operand B in hex based on the IEEE 754 half precision format and press key 3. This will display B on the Hex display
3. Finally the Result will be displayed on the Hex display after pressing key 4 on the hex board.

Description of any added features:

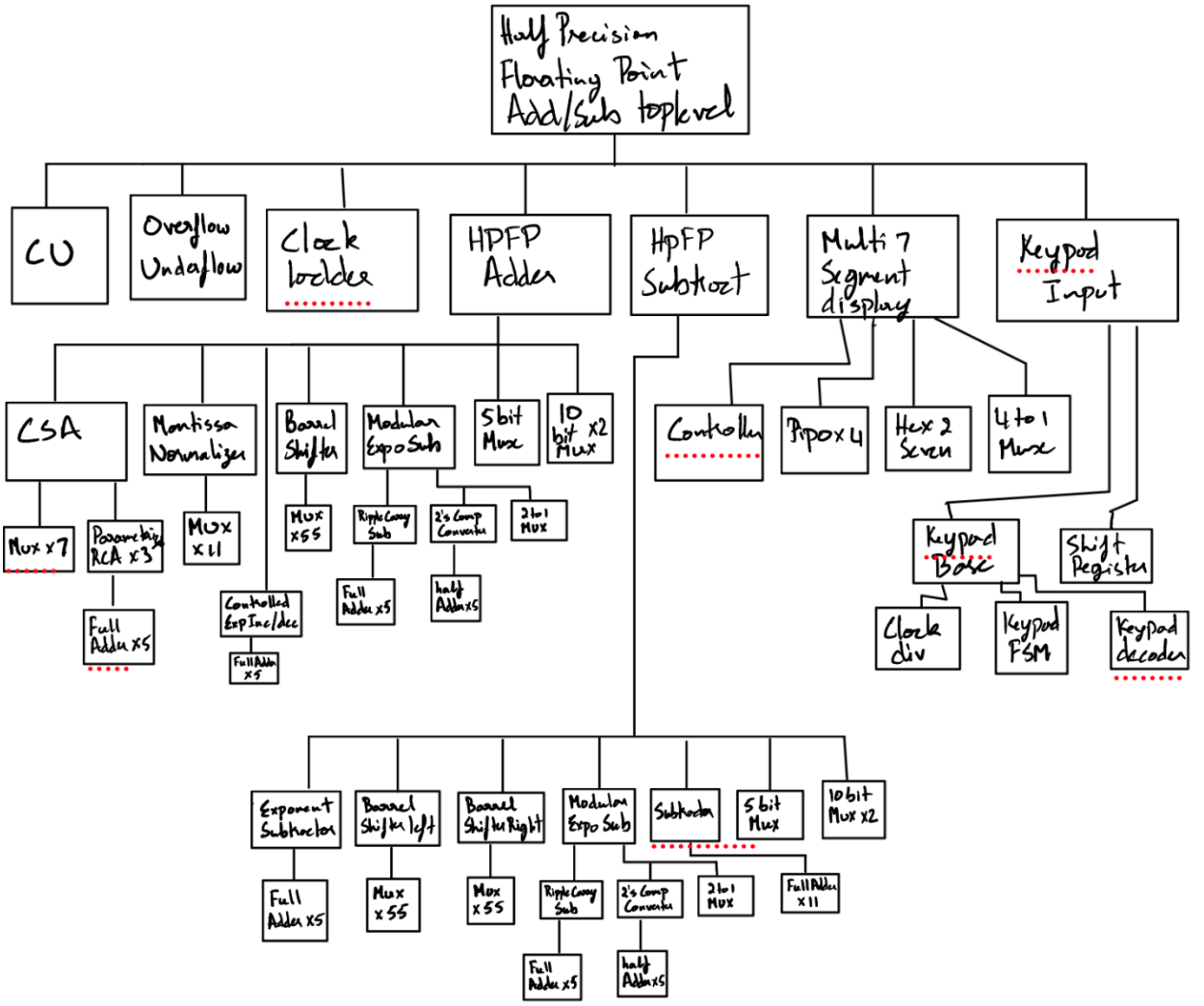
No Additional Added features

-BONUS: Early Submission

-BONUS: Subtraction

-BONUS: Overflow and Underflow Flags

Organization Diagram:



Test Results:

- **Summary:**

A	Hex value of A	B	Hex value of B	Result (R)	Hex value of R	Overflow	Underflow
3.75	0x4380	5.125	0x4520	8.875	0x4870	0	0
3.75	0x4380	-5.125	0xC520	-1.375	0xBD80	0	0
-3.75	0xC380	5.125	0x4520	1.375	0x3D80	0	0
9.5	0x48C0	0.5625	0x3880	10.0625	0x4908	0	0
9.5	0x48C0	-0.5625	0xB880	8.9375	0x4878	0	0
-9.5	0xC8C0	-0.5625	0xB880	-10.0625	0xC908	0	0
125.125	0x57D2	-12.0625	0xCA08	113.0625	0x5711	0	0
1000	0x63D0	1049	0x6419	2050	0x6801	0	0
65504	0x7BFF	65504	0x7BFF	+∞	0x7C00	1	0
-65504	0xFBFF	-65504	0xFBFF	-∞	0xFC00	1	0
0.0000610948	0x0401	-0.00006103515625	0x8400	+0	0x0000	0	1
0.000244140625	0x0C00	-0.00024437904358	0x8C01	-0	0x8000	0	1

- **Demonstration**

https://youtu.be/Am9OUKdBFNQ?si=iWIaEgJJ6qxy8XxN

SystemVerilog Code:

```
1   //Half Precision FLoating Point Adder/Subtractor Top Level module - DE10 Implementaton
2   module HalfPrecisionFPAdder
3   (
4      input clk,
5      input reset,
6      input [3:0] row,
7      output [3:0] col,
8      output logic [3:0] CAT,
9      output logic [0:6] SEG,
10     input key2,key3,key4,
11     output Overflow, Underflow
12  );
13
14     logic [15:0] BCDSM,Result;
15     logic [31:0] Y;
16     logic MuxRate, CounterCLK;
17     logic [3:0] HEX0, HEX1, HEX2, HEX3;
18     logic value, trig;
19     logic [4:0] Exponent;
20     logic [9:0] Mantissa;
21     logic [15:0] A_reg, B_reg;
22
23
24     keypad_input #(.DIGITS(4)) keypad_input_inst
25     (
26        .clk(clk),
27        .reset(reset),
28        .row(row),
29        .col(col),
30        .out(BCDSM),
31        .value(value),
32        .trig(trig)
33     );
34
```

```
34
35
36     ClockLadder ClockLadder_inst
37     (
38        .clock(clk) ,  // input  clock_sig
39        .clear(reset) ,   // input  clear_sig
40        .Y(Y)    // output [31:0] Y_sig
41     );
42     assign CounterCLK = Y[22];
43     assign MuxRate = Y[17];
44
45     logic [1:0] displaysel;
46
47     always_ff @(posedge clk, negedge reset) begin
48         if(!reset) begin
49             A_reg <= 0;
50             B_reg <= 0;
51             displaysel <= 2'b11;
52         end
53         else if(!key2) begin
54             A_reg <= BCDSM;
55             displaysel <= 2'b00;
56         end
57         else if(!key3) begin
58             B_reg <= BCDSM;
59             displaysel <= 2'b01;
60         end
61         else if(!key4) begin
62             displaysel <= 2'b10;
63         end
64     end
```

```
65
66      logic [15:0] Aout, Bout;
67      logic addsub;
68      logic sign;
69      CU CU_inst
70      (
71          .A(A_reg) , // input [15:0] A_sig
72          .B(B_reg) , // input [15:0] B_sig
73          .Aout(Aout) ,  // output [15:0] Aout_sig
74          .Bout(Bout) ,  // output [15:0] Bout_sig
75          .addsub(addsub),  // output  addsub_sig
76          .sign(sign)
77      );
78
79      logic [4:0] ExponentSub, ExponentAdd;
80      logic [9:0] MantissaAdd, MantissaSub;
81      logic UA;
82      HPFPSubtractor HPFPSubtractor_inst
83      (
84          .A(Aout) ,  // input [15:0] A_sig
85          .B(Bout) ,  // input [15:0] B_sig
86          .Exponent(ExponentSub) ,   // output [4:0] Exponent_sig
87          .Mantissa(MantissaSub),    // output [9:0] Mantissa_sig
88          .exponentsubtractorcout(exponentsubtractorcout)
89      );
90
91      HPFPAdder HPFPAdder_inst
92      (
93          .A(Aout) ,  // input [15:0] A_sig
94          .B(Bout) ,  // input [15:0] B_sig
95          .Exponent(ExponentAdd) ,   // output [4:0] Exponent_sig
96          .Mantissa(MantissaAdd)  // output [9:0] Mantissa_sig
97      );
98
```

```
97      );
98
99      assign Exponent = addsub ? ExponentSub : ExponentAdd;
100     assign Mantissa = addsub ? MantissaSub : MantissaAdd;
101     logic OE,UE;
102
103     OverflowUnderflow OverflowUnderflow_inst
104     (
105         .EA(A_reg[14:10]) ,  // input [4:0] EA_sig
106         .EB(B_reg[14:10]) ,  // input [4:0] EB_sig
107         .finalmantissa(Mantissa) , // input [9:0] finalmantissa_sig
108         .finalexponent(Exponent) , // input [5:0] finalexponent_sig
109         .exponentsubtractorcout(exponentsubtractorcout) ,  // input  exponentsubtractorcout_sig
110         .Overflow(OE) ,   // output  Overflow_sig
111         .Underflow(UE)    // output  Underflow_sig
112     );
113
114     assign Overflow  = addsub ? 1'b0 : OE;
115     assign Underflow = addsub ? UE : 1'b0;
116
117     logic [15:0] Resulttemp;
118
119     always_comb begin
120         if(Overflow)
121             Resulttemp = {sign,5'b11111,10'b0000000000};
122         else if (Underflow)
123             Resulttemp = {sign,5'b00000,10'b0000000000};
124         else
125             Resulttemp = {sign,Exponent,Mantissa};
126     end
127
128
```

```
127
128
129         assign Result = Resulttemp;
130         logic [15:0] display;
131
132         always_comb begin
133            case(displaysel)
134               2'b00: display = A_reg;
135               2'b01: display = B_reg;
136               2'b10: display = Result;
137               default: display = BCDSM;
138            endcase
139         end
140
141
142         assign HEX0 = display[3:0];
143         assign HEX1 = display[7:4];
144         assign HEX2 = display[11:8];
145         assign HEX3 = display[15:12];
146
147         Multi7segdisplay Multi7segdisplay_inst
148         (
149            .HEX0(HEX0) ,  // input [3:0] HEX0_sig
150            .HEX1(HEX1) ,  // input [3:0] HEX1_sig
151            .HEX2(HEX2) ,  // input [3:0] HEX2_sig
152            .HEX3(HEX3) ,  // input [3:0] HEX3_sig
153            .CLK(MuxRate) ,   // input  CLK_sig
154            .Reset(reset) ,   // input  Reset_sig
155            .Load(CounterCLK) ,  // input  Load_sig
156            .SEG(SEG) , // output [0:6] SEG_sig
157            .CAT(CAT)   // output [3:0] CAT_sig
158         );
159
160      endmodule
```

```
1     //Control Module to decide addition or Subtraction based on the sign bit
2     module CU
3     (
4        input [15:0] A,B,
5        output [15:0] Aout, Bout,
6        output addsub,
7        output sign
8     );
9        assign addsub = A[15] ^ B[15];
10
11       always_comb begin
12          if(addsub) begin
13             if(A[14:0] > B[14:0]) begin
14                Aout = A;
15                Bout = B;
16                sign = A[15];
17             end
18             else begin
19                Aout = B;
20                Bout = A;
21                sign = B[15];
22             end
23          end
24          else begin
25             Aout = A;
26             Bout = B;
27             sign = A[15];
28          end
29       end
30
31    endmodule
```

```
1   //Clock Ladder
2   module ClockLadder
3   (
4      input clock, clear,
5      output logic [31:0] Y
6   );
7      always_ff @ (posedge clock, negedge clear)
8         if(~clear) Y <= 1'b0;
9            else Y <= Y + 1'b1;
10  endmodule
11
```

```
1   //OverFlow and UnderFlow Flag module
2   module OverflowUnderflow
3   (
4      input [4:0] EA, EB,
5      input [9:0] finalmantissa,
6      input [5:0] finalexponent,
7      input exponentsubtractorcout,
8      output Overflow, Underflow
9   );
10     logic Exception;
11     logic zero;
12     assign Exception = (&EA) | (&EB);
13     assign zero = Exception ? 1'b0 : (finalmantissa == 10'd0) ? 1'b1 : 1'b0;
14     assign Overflow = (finalexponent > 5'b11110) && !zero;
15     assign Underflow = ~exponentsubtractorcout;
16
17  endmodule
```

```
1   //Half Precision Floating Point Adder
2   module HPFPAdder
3   (
4      input [15:0] A,B,
5      output [4:0] Exponent,
6      output [9:0] Mantissa
7   );
8
9      logic [4:0] modexposubout, expoincmuxout;
10     logic modexpocout;
11     logic [9:0] smuxout, amuxout;
12     logic [10:0]  shiftermuxout, mantissarsout, addermuxout;
13     logic [10:0] adderout;
14     logic addercout;
15
16
17     ModularExponentSub #(.N(5)) ModularExponentSub_inst
18     (
19        .A(B[14:10]) , // input [(N-1):0] A_sig
20        .B(A[14:10]) , // input [(N-1):0] B_sig
21        .out(modexposubout) ,   // output [(N-1):0] out_sig
22        .cout(modexpocout)   // output  cout_sig
23     );
24
25     tenbitmux tenbitmux_shiftermux
26     (
27        .A(A[9:0]) ,   // input [9:0] A_sig
28        .B(B[9:0]) ,   // input [9:0] B_sig
29        .select(modexpocout) ,  // input  select_sig
30        .out(smuxout)  // output [9:0] out_sig
31     );
```

```
    );
    assign shiftermuxout = {1'b1,smuxout};

    tenbitmux tenbitmux_addermux
    (
        .A(B[9:0]) ,   // input [9:0] A_sig
        .B(A[9:0]) ,   // input [9:0] B_sig
        .select(modexpocout) ,  // input  select_sig
        .out(amuxout)  // output [9:0] out_sig
    );
    assign addermuxout = {1'b1,amuxout};


    fivebitmux expoincmux
    (
        .A(B[14:10]) , // input [4:0] A_sig
        .B(A[14:10]) , // input [4:0] B_sig
        .select(modexpocout) ,  // input  select_sig
        .out(expoincmuxout)  // output [4:0] out_sig
    );

    BarrelShifter MantissaRightShifter
    (
        .A(shiftermuxout) ,  // input [10:0] A_sig
        .B(modexposubout) ,  // input [4:0] B_sig
        .Y(mantissarsout)    // output [10:0] Y_sig
    );


    CSA CSA_inst
    (
        .A(mantissarsout) ,  // input [10:0] A_sig
        .B(addermuxout) , // input [10:0] B_sig
        .Cin(1'b0) ,    // input  Cin_sig
        .Sum(adderout) ,  // output [10:0] Sum_sig
        .Cout(addercout)  // output  Cout_sig
    );

    logic select;

    always_comb begin
        if(addercout)
            select = 1'b1;
        else
            select = 1'b0;
        end

    ControlledExponent_INC_DEC ControlledExponent_INC_DEC_inst
    (
        .A(expoincmuxout) ,  // input [4:0] A_sig
        .select(select) , // input  select_sig
        .cin(~addercout) ,   // input  cin_sig
        .exponent(Exponent)  // output [4:0] exponent_sig
    );


    MantissaNormalizer MantissaNormalizer_inst
    (
        .A(adderout) , // input [10:0] A_sig
        .select(addercout) , // input  select_sig
        .mantissa(Mantissa)  // output [9:0] mantissa_sig
    );

endmodule
```

```
1   //Carry Select Adder
2   module CSA
3   (
4       input [10:0] A,B,
5       input Cin,
6       output [10:0] Sum,
7       output Cout
8   );
9
10
11      wire [4:0] lower;
12      wire [5:0] RCA2, RCA3, upper;
13      wire C80, C81,C4;
14
15      ParameterizedRCA #(.N(5))ParameterizedRCA_inst0
16      (
17          .Cin(Cin) , // input  Cin_sig
18          .A(A[4:0]) ,   // input [(N-1):0] A_sig
19          .B(B[4:0]) ,   // input [(N-1):0] B_sig
20          .Sum(lower[4:0]) ,   // output [(N-1):0] Sum_sig
21          .Cout(C4)   // output  Cout_sig
22      );
23
24      ParameterizedRCA #(.N(6))ParameterizedRCA_inst1
25      (
26          .Cin(1'b0) ,   // input  Cin_sig
27          .A(A[10:5]) ,  // input [(N-1):0] A_sig
28          .B(B[10:5]) ,  // input [(N-1):0] B_sig
29          .Sum(RCA2) ,   // output [(N-1):0] Sum_sig
30          .Cout(C80)  // output  Cout_sig
31      );
32
33      ParameterizedRCA #(.N(6))ParameterizedRCA_inst2
34      (
35          .Cin(1'b1) ,   // input  Cin_sig
36          .A(A[10:5]) ,  // input [(N-1):0] A_sig
37          .B(B[10:5]) ,  // input [(N-1):0] B_sig
38          .Sum(RCA3) ,   // output [(N-1):0] Sum_sig
39          .Cout(C81)  // output  Cout_sig
40      );
41
42      Mux R5_inst
43      (
44          .A(RCA3[0]) ,  // input  A_sig
45          .B(RCA2[0]) ,  // input  B_sig
46          .S(C4) , // input  C4_sig
47          .Out(upper[0])    // output  R_sig
48      );
49
50      Mux R6_inst
51      (
52          .A(RCA3[1]) ,  // input  A_sig
53          .B(RCA2[1]) ,  // input  B_sig
54          .S(C4) , // input  C4_sig
55          .Out(upper[1])    // output  R_sig
56      );
57
```

```
    Mux R7_inst
    (
        .A(RCA3[2]) ,  // input  A_sig
        .B(RCA2[2]) ,  // input  B_sig
        .S(C4) , // input  C4_sig
        .Out(upper[2])    // output  R_sig
    );

    Mux R8_inst
    (
        .A(RCA3[3]) ,  // input  A_sig
        .B(RCA2[3]) ,  // input  B_sig
        .S(C4) , // input  C4_sig
        .Out(upper[3])    // output  R_sig
    );

    Mux R9_inst
    (
        .A(RCA3[4]) ,  // input  A_sig
        .B(RCA2[4]) ,  // input  B_sig
        .S(C4) , // input  C4_sig
        .Out(upper[4])    // output  R_sig
    );

    Mux R10_inst
    (
        .A(RCA3[5]) ,  // input  A_sig
        .B(RCA2[5]) ,  // input  B_sig
        .S(C4) , // input  C4_sig
        .Out(upper[5])    // output  R_sig
    );
```

```
    Mux R10_inst
    (
        .A(RCA3[5]) ,  // input  A_sig
        .B(RCA2[5]) ,  // input  B_sig
        .S(C4) , // input  C4_sig
        .Out(upper[5])    // output  R_sig
    );

    Mux Cout_inst
    (
        .A(C81) ,   // input  A_sig
        .B(C80) ,   // input  B_sig
        .S(C4) , // input  C4_sig
        .Out(Cout)  // output  R_sig
    );

    assign Sum = {upper,lower};
endmodule
```

```
//Mux
module Mux(

    input A,
    input B,
    input S,
    output Out
);

    assign Out = S ? A : B;

endmodule

```

```
//Paramterized Ripple Carry Adder
module ParameterizedRCA #(parameter N = 8)
(
   input Cin,
   input [N-1:0] A,B,
   output [N-1:0] Sum,
   output Cout
);

   wire [N:0] C;
   assign C[0] = Cin;
   assign Cout = C[N];

   genvar i;
   generate
      for(i=0;i<N;i=i+1)
         begin:full_adder
            FullAdder FullAdder_inst (A[i],B[i],C[i],Sum[i],C[i+1]);
         end
   endgenerate

endmodule
```

```
//FullAdder
module FullAdder(
   input ai,bi, cini,
   output logic si,couti);
      always_comb
         case({ai,bi,cini})
            3'b000: begin couti = 1'b0; si = 1'b0; end
            3'b001: begin couti = 1'b0; si = 1'b1; end
            3'b010: begin couti = 1'b0; si = 1'b1; end
            3'b011: begin couti = 1'b1; si = 1'b0; end
            3'b100: begin couti = 1'b0; si = 1'b1; end
            3'b101: begin couti = 1'b1; si = 1'b0; end
            3'b110: begin couti = 1'b1; si = 1'b0; end
            3'b111: begin couti = 1'b1; si = 1'b1; end
         endcase
endmodule
```

```
//Controlled Exponent Increment and Decrement(Top Level)
module ControlledExponent_INC_DEC
(
   input [4:0] A,
   input select,cin,
   output [4:0] exponent
);

   logic [5:0] C;
   assign C[0] = cin;

   FullAdder FullAdder_instS0(.ai(A[0]) ,.bi(select ^ cin) ,.cini(C[0]) ,.si(exponent[0]) ,.couti(C[1]));
   FullAdder FullAdder_instS1(.ai(A[1]) ,.bi(cin ^ 1'b0) ,.cini(C[1]) ,.si(exponent[1]) ,.couti(C[2]));
   FullAdder FullAdder_instS2(.ai(A[2]) ,.bi(cin ^ 1'b0) ,.cini(C[2]) ,.si(exponent[2]) ,.couti(C[3]));
   FullAdder FullAdder_instS3(.ai(A[3]) ,.bi(cin ^ 1'b0) ,.cini(C[3]) ,.si(exponent[3]) ,.couti(C[4]));
   FullAdder FullAdder_instS4(.ai(A[4]) ,.bi(cin ^ 1'b0) ,.cini(C[4]) ,.si(exponent[4]) ,.couti(C[5]));

endmodule
```

```
//Mantissa Normalizer
module MantissaNormalizer
(
   input [10:0] A,
   input select,
   output [9:0] mantissa
);

   logic [10:0] U;
   logic [9:0] Y;
   Mux Mux_inst0(.A(A[1]) ,.B(A[0]) ,.S(select) ,.Out(U[0]));
   Mux Mux_inst1(.A(A[2]) ,.B(A[1]) ,.S(select) ,.Out(U[1]));
   Mux Mux_inst2(.A(A[3]) ,.B(A[2]) ,.S(select) ,.Out(U[2]));
   Mux Mux_inst3(.A(A[4]) ,.B(A[3]) ,.S(select) ,.Out(U[3]));
   Mux Mux_inst4(.A(A[5]) ,.B(A[4]) ,.S(select) ,.Out(U[4]));
   Mux Mux_inst5(.A(A[6]) ,.B(A[5]) ,.S(select) ,.Out(U[5]));
   Mux Mux_inst6(.A(A[7]) ,.B(A[6]) ,.S(select) ,.Out(U[6]));
   Mux Mux_inst7(.A(A[8]) ,.B(A[7]) ,.S(select) ,.Out(U[7]));
   Mux Mux_inst8(.A(A[9]) ,.B(A[8]) ,.S(select) ,.Out(U[8]));
   Mux Mux_inst9(.A(A[10]) ,.B(A[9]) ,.S(select) ,.Out(U[9]));
   Mux Mux_instA(.A(1'b0) ,.B(A[10]) ,.S(select) ,.Out(U[10]));

   always_comb begin
      if(select == 1) begin
         if(A[0] == 1) begin
            Y = U[9:0] + 1'b1;
         end
         else begin
            Y = U[9:0];
         end
      end
      else begin
         Y = A[9:0];
      end
   end

   assign mantissa = Y;

endmodule
```

```
//Right Barrel Shifter
module BarrelShifter (

   input [10:0] A,
   input [4:0] B,
   output [10:0] Y
);


   logic [10:0] V,U,W,X;

   //B4
   Mux Mux_inst0(.A(1'b0) ,.B(A[0]) ,.S(B[4]) ,.Out(U[0]));
   Mux Mux_inst1(.A(1'b0) ,.B(A[1]) ,.S(B[4]) ,.Out(U[1]));
   Mux Mux_inst2(.A(1'b0) ,.B(A[2]) ,.S(B[4]) ,.Out(U[2]));
   Mux Mux_inst3(.A(1'b0) ,.B(A[3]) ,.S(B[4]) ,.Out(U[3]));
   Mux Mux_inst4(.A(1'b0) ,.B(A[4]) ,.S(B[4]) ,.Out(U[4]));
   Mux Mux_inst5(.A(1'b0) ,.B(A[5]) ,.S(B[4]) ,.Out(U[5]));
   Mux Mux_inst6(.A(1'b0) ,.B(A[6]) ,.S(B[4]) ,.Out(U[6]));
   Mux Mux_inst7(.A(1'b0) ,.B(A[7]) ,.S(B[4]) ,.Out(U[7]));
   Mux Mux_inst8(.A(1'b0) ,.B(A[8]) ,.S(B[4]) ,.Out(U[8]));
   Mux Mux_inst9(.A(1'b0) ,.B(A[9]) ,.S(B[4]) ,.Out(U[9]));
   Mux Mux_instA(.A(1'b0) ,.B(A[10]) ,.S(B[4]) ,.Out(U[10]));

```

```
24
25      //B3
26      Mux Mux_instB(.A(U[8]) ,.B(U[0]) ,.S(B[3]) ,.Out(V[0]));
27      Mux Mux_instC(.A(U[9]) ,.B(U[1]) ,.S(B[3]) ,.Out(V[1]));
28      Mux Mux_instD(.A(U[10]) ,.B(U[2]) ,.S(B[3]) ,.Out(V[2]));
29      Mux Mux_instE(.A(1'b0) ,.B(U[3]) ,.S(B[3]) ,.Out(V[3]));
30      Mux Mux_instF(.A(1'b0) ,.B(U[4]) ,.S(B[3]) ,.Out(V[4]));
31      Mux Mux_inst00(.A(1'b0) ,.B(U[5]) ,.S(B[3]) ,.Out(V[5]));
32      Mux Mux_inst01(.A(1'b0) ,.B(U[6]) ,.S(B[3]) ,.Out(V[6]));
33      Mux Mux_inst02(.A(1'b0) ,.B(U[7]) ,.S(B[3]) ,.Out(V[7]));
34      Mux Mux_inst03(.A(1'b0) ,.B(U[8]) ,.S(B[3]) ,.Out(V[8]));
35      Mux Mux_inst04(.A(1'b0) ,.B(U[9]) ,.S(B[3]) ,.Out(V[9]));
36      Mux Mux_inst05(.A(1'b0) ,.B(U[10]) ,.S(B[3]) ,.Out(V[10]));
37
38      //B2
39      Mux Mux_inst06(.A(V[4]) ,.B(V[0]) ,.S(B[2]) ,.Out(W[0]));
40      Mux Mux_inst07(.A(V[5]) ,.B(V[1]) ,.S(B[2]) ,.Out(W[1]));
41      Mux Mux_inst08(.A(V[6]) ,.B(V[2]) ,.S(B[2]) ,.Out(W[2]));
42      Mux Mux_inst09(.A(V[7]) ,.B(V[3]) ,.S(B[2]) ,.Out(W[3]));
43      Mux Mux_inst0A(.A(V[8]) ,.B(V[4]) ,.S(B[2]) ,.Out(W[4]));
44      Mux Mux_inst0B(.A(V[9]) ,.B(V[5]) ,.S(B[2]) ,.Out(W[5]));
45      Mux Mux_inst0C(.A(V[10]) ,.B(V[6]) ,.S(B[2]) ,.Out(W[6]));
46      Mux Mux_inst0D(.A(1'b0) ,.B(V[7]) ,.S(B[2]) ,.Out(W[7]));
47      Mux Mux_inst0E(.A(1'b0) ,.B(V[8]) ,.S(B[2]) ,.Out(W[8]));
48      Mux Mux_inst0F(.A(1'b0) ,.B(V[9]) ,.S(B[2]) ,.Out(W[9]));
49      Mux Mux_inst10(.A(1'b0) ,.B(V[10]) ,.S(B[2]) ,.Out(W[10]));
50
```

```
50
51      //B1
52      Mux Mux_inst11(.A(W[2]) ,.B(W[0]) ,.S(B[1]) ,.Out(X[0]));
53      Mux Mux_inst12(.A(W[3]) ,.B(W[1]) ,.S(B[1]) ,.Out(X[1]));
54      Mux Mux_inst13(.A(W[4]) ,.B(W[2]) ,.S(B[1]) ,.Out(X[2]));
55      Mux Mux_inst14(.A(W[5]) ,.B(W[3]) ,.S(B[1]) ,.Out(X[3]));
56      Mux Mux_inst15(.A(W[6]) ,.B(W[4]) ,.S(B[1]) ,.Out(X[4]));
57      Mux Mux_inst16(.A(W[7]) ,.B(W[5]) ,.S(B[1]) ,.Out(X[5]));
58      Mux Mux_inst17(.A(W[8]) ,.B(W[6]) ,.S(B[1]) ,.Out(X[6]));
59      Mux Mux_inst18(.A(W[9]) ,.B(W[7]) ,.S(B[1]) ,.Out(X[7]));
60      Mux Mux_inst19(.A(W[10]) ,.B(W[8]) ,.S(B[1]) ,.Out(X[8]));
61      Mux Mux_inst1A(.A(1'b0) ,.B(W[9]) ,.S(B[1]) ,.Out(X[9]));
62      Mux Mux_inst1B(.A(1'b0) ,.B(W[10]) ,.S(B[1]) ,.Out(X[10]));
63
64      //B0
65      Mux Mux_inst1C(.A(X[1]) ,.B(X[0]) ,.S(B[0]) ,.Out(Y[0]));
66      Mux Mux_inst1D(.A(X[2]) ,.B(X[1]) ,.S(B[0]) ,.Out(Y[1]));
67      Mux Mux_inst1E(.A(X[3]) ,.B(X[2]) ,.S(B[0]) ,.Out(Y[2]));
68      Mux Mux_inst1F(.A(X[4]) ,.B(X[3]) ,.S(B[0]) ,.Out(Y[3]));
69      Mux Mux_inst100(.A(X[5]) ,.B(X[4]) ,.S(B[0]) ,.Out(Y[4]));
70      Mux Mux_inst101(.A(X[6]) ,.B(X[5]) ,.S(B[0]) ,.Out(Y[5]));
71      Mux Mux_inst102(.A(X[7]) ,.B(X[6]) ,.S(B[0]) ,.Out(Y[6]));
72      Mux Mux_inst103(.A(X[8]) ,.B(X[7]) ,.S(B[0]) ,.Out(Y[7]));
73      Mux Mux_inst104(.A(X[9]) ,.B(X[8]) ,.S(B[0]) ,.Out(Y[8]));
74      Mux Mux_inst105(.A(X[10]) ,.B(X[9]) ,.S(B[0]) ,.Out(Y[9]));
75      Mux Mux_inst106(.A(1'b0) ,.B(X[10]) ,.S(B[0]) ,.Out(Y[10]));
76
77  endmodule
```

```
//Modular Exponent Subtractor(Top Level)
module ModularExponentSub #(parameter N = 8)
(
    input [N-1:0] A, B,
    output [N-1:0] out,
    output cout
);

    logic [N-1:0] R,Rcomp;
    logic Cout;

    RippleCarrySub #(.N(N)) RippleCarrySub_inst
    (
        .A(A) ,  // input [(N-1):0] A_sig
        .B(B^{N{1'b1}}) , // input [(N-1):0] B_sig
        .cin(1'b1) ,
        .R(R) ,  // output [(N-1):0] R_sig
        .Cout(cout)    // output  Cout_sig
    );
    TwosCompConverter #(.N(N)) TwosCompConverter_inst
    (
        .A(R) ,  // input [(N-1):0] A_sig
        .B(Rcomp)   // output [(N-1):0] B_sig
    );
    twotoonemux #(.N(N))twotoonemux_inst
    (
        .A(R) ,  // input [(N-1):0] A_sig
        .B(Rcomp) , // input [(N-1):0] B_sig
        .select(cout) ,   // input  select_sig
        .out(out)   // output [(N-1):0] out_sig
    );

endmodule
```

```
//Ripple Carry Subtractor
module RippleCarrySub #(parameter N = 8)
(
    input [N-1:0] A, B,
    input cin,
    output [N-1:0] R,
    output Cout
);

    logic [N:0] C;
    assign C[0] = cin;
    assign Cout = C[N];

    genvar i;
    generate
       for(i=0;i<N;i=i+1)
          begin:full_adder
             FullAdder FullAdder_inst (A[i],B[i],C[i],R[i],C[i+1]);
          end
    endgenerate

endmodule
```

```
1   //Twos comp converter
2   module TwosCompConverter #(parameter N = 8)
3   (
4       input[N-1:0]A,
5       output[N-1:0]B
6   );
7       wire [N:0] ha;
8       assign ha[0] = 1'b1;
9          genvar i;
10         generate
11            for(i=0;i<N;i=i+1)
12               begin: twosFor
13                  halfADDER halfADDER_inst1
14                  (
15                     .s(B[i]),
16                     .cout(ha[i+1]),
17                     .a((A[i])^(1'b1)),
18                     .b(ha[i])
19                  );
20               end
21         endgenerate
22  endmodule
23
```

```
1   //Half Adder
2   module halfADDER(s,cout,a,b);
3      input a,b;
4      output s, cout;
5      and(cout,a,b);
6      xor(s,a,b);
7   endmodule
8
```

```
1   //Two to one mux
2   module twotoonemux #(parameter N = 8)
3   (
4       input [N-1:0] A, B,
5       input select,
6       output [N-1:0] out
7   );
8
9       always_comb begin
10         if(select)
11            out = A;
12         else
13            out = B;
14      end
15
16  endmodule
```

```
1   //5 bit mux
2   module fivebitmux
3   (
4       input [4:0] A,B,
5       input select,
6       output [4:0] out
7   );
8
9       assign out = select ? A : B;
10
11  endmodule
12
```

```
//10 bit mux
module tenbitmux
(
   input [9:0] A,B,
   input select,
   output [9:0] out
);

   assign out = select ? A : B;

endmodule
```

```
//Half precision Floating Point Subtractor
module HPFPSubtractor
(
   input [15:0] A,B,
   output [4:0] Exponent,
   output [9:0] Mantissa,
   output exponentsubtractorcout
);

   logic [4:0] modexposubout, expoincmuxout;
   logic modexpocout;
   logic [9:0] smuxout, amuxout;
   logic [10:0]  shiftermuxout, mantissarsout, addermuxout;
   logic [10:0] adderout;
   logic addercout;


   ModularExponentSub #(.N(5)) ModularExponentSub_inst
   (
      .A(A[14:10]) , // input [(N-1):0] A_sig
      .B(B[14:10]) , // input [(N-1):0] B_sig
      .out(modexposubout) ,   // output [(N-1):0] out_sig
      .cout(modexpocout)   // output  cout_sig
   );

   tenbitmux tenbitmux_shiftermux
   (
      .A(B[9:0]) ,   // input [9:0] A_sig
      .B(A[9:0]) ,   // input [9:0] B_sig
      .select(modexpocout) ,  // input  select_sig
      .out(smuxout)  // output [9:0] out_sig
   );
   assign shiftermuxout = {1'b1,smuxout};

```

```
34
35          tenbitmux tenbitmux_addermux
36          (
37              .A(A[9:0]) ,   // input [9:0] A_sig
38              .B(B[9:0]) ,   // input [9:0] B_sig
39              .select(modexpocout) ,  // input  select_sig
40              .out(amuxout)  // output [9:0] out_sig
41          );
42          assign addermuxout = {1'b1,amuxout};
43
44
45          fivebitmux expoincmux
46          (
47              .A(A[14:10]) , // input [4:0] A_sig
48              .B(B[14:10]) , // input [4:0] B_sig
49              .select(modexpocout) ,  // input  select_sig
50              .out(expoincmuxout)  // output [4:0] out_sig
51          );
52
53          BarrelShifter MantissaRightShifter
54          (
55              .A(shiftermuxout) ,  // input [10:0] A_sig
56              .B(modexposubout) ,  // input [4:0] B_sig
57              .Y(mantissarsout)    // output [10:0] Y_sig
58          );
59
60          Subtractor Subtractor_inst
61          (
62              .A(addermuxout) , // input [10:0] A_sig
63              .B(mantissarsout) ,  // input [10:0] B_sig
64              .cin(1'b1) ,   // input  cin_sig
65              .cout(addercout) ,    // output  cout_sig
66              .R(adderout)   // output [10:0] R_sig
67          );
68
69          logic [3:0] leadingzeros;
70
71          always_comb begin
72              leadingzeros = 4'd11;
73              for (integer i = 10; i >= 0; i = i - 1) begin
74                  if (adderout[i] && leadingzeros == 4'd11) begin
75                      leadingzeros = 4'd11 - (i+1'b1);
76                  end
77              end
78          end
79          ExponentSubtractor ExponentSubtractor_inst
80          (
81              .A(expoincmuxout) ,  // input [4:0] A_sig
82              .B(leadingzeros) ,   // input [3:0] B_sig
83              .exponent(Exponent),    // output [4:0] exponent_sig
84              .cout(exponentsubtractorcout)
85          );
86
87          BarrelShifterLeft MantissaNormalizerLeft_inst
88          (
89              .A(adderout) , // input [10:0] A_sig
90              .B(leadingzeros) ,   // input [4:0] B_sig
91              .Y(Mantissa)   // output [10:0] Y_sig
92          );
93
94      endmodule
```

```
//Exponent Decrementer
module ExponentSubtractor
(
    input [4:0] A,
    input [3:0] B,
    output [4:0] exponent,
    output cout
);


    logic [5:0] C;
    assign C[0] = 1'b1;

    FullAdder FullAdder_instS0(.ai(A[0]) ,.bi(B[0] ^ 1'b1) ,.cini(C[0]) ,.si(exponent[0]) ,.couti(C[1]));
    FullAdder FullAdder_instS1(.ai(A[1]) ,.bi(B[1] ^ 1'b1) ,.cini(C[1]) ,.si(exponent[1]) ,.couti(C[2]));
    FullAdder FullAdder_instS2(.ai(A[2]) ,.bi(B[2] ^ 1'b1) ,.cini(C[2]) ,.si(exponent[2]) ,.couti(C[3]));
    FullAdder FullAdder_instS3(.ai(A[3]) ,.bi(B[3] ^ 1'b1) ,.cini(C[3]) ,.si(exponent[3]) ,.couti(C[4]));
    FullAdder FullAdder_instS4(.ai(A[4]) ,.bi(1'b1) ,.cini(C[4]) ,.si(exponent[4]) ,.couti(C[5]));
    assign cout = C[5];

endmodule
```

```
//Left Barrel Shifter
module BarrelShifterLeft (

    input [10:0] A,
    input [4:0] B,
    output [9:0] Y
);


    logic [10:0] V,U,W,X,T;

    //B4
    Mux Mux_inst0(.A(1'b0) ,.B(A[0]) ,.S(B[4]) ,.Out(U[0]));
    Mux Mux_inst1(.A(1'b0) ,.B(A[1]) ,.S(B[4]) ,.Out(U[1]));
    Mux Mux_inst2(.A(1'b0) ,.B(A[2]) ,.S(B[4]) ,.Out(U[2]));
    Mux Mux_inst3(.A(1'b0) ,.B(A[3]) ,.S(B[4]) ,.Out(U[3]));
    Mux Mux_inst4(.A(1'b0) ,.B(A[4]) ,.S(B[4]) ,.Out(U[4]));
    Mux Mux_inst5(.A(1'b0) ,.B(A[5]) ,.S(B[4]) ,.Out(U[5]));
    Mux Mux_inst6(.A(1'b0) ,.B(A[6]) ,.S(B[4]) ,.Out(U[6]));
    Mux Mux_inst7(.A(1'b0) ,.B(A[7]) ,.S(B[4]) ,.Out(U[7]));
    Mux Mux_inst8(.A(1'b0) ,.B(A[8]) ,.S(B[4]) ,.Out(U[8]));
    Mux Mux_inst9(.A(1'b0) ,.B(A[9]) ,.S(B[4]) ,.Out(U[9]));
    Mux Mux_instA(.A(1'b0) ,.B(A[10]) ,.S(B[4]) ,.Out(U[10]));

    //B3
    Mux Mux_instB(.A(1'b0) ,.B(U[0]) ,.S(B[3]) ,.Out(V[0]));
    Mux Mux_instC(.A(1'b0) ,.B(U[1]) ,.S(B[3]) ,.Out(V[1]));
    Mux Mux_instD(.A(1'b0) ,.B(U[2]) ,.S(B[3]) ,.Out(V[2]));
    Mux Mux_instE(.A(1'b0) ,.B(U[3]) ,.S(B[3]) ,.Out(V[3]));
    Mux Mux_instF(.A(1'b0) ,.B(U[4]) ,.S(B[3]) ,.Out(V[4]));
    Mux Mux_inst00(.A(1'b0) ,.B(U[5]) ,.S(B[3]) ,.Out(V[5]));
    Mux Mux_inst01(.A(1'b0) ,.B(U[6]) ,.S(B[3]) ,.Out(V[6]));
    Mux Mux_inst02(.A(1'b0) ,.B(U[7]) ,.S(B[3]) ,.Out(V[7]));
    Mux Mux_inst03(.A(U[0]) ,.B(U[8]) ,.S(B[3]) ,.Out(V[8]));
    Mux Mux_inst04(.A(U[1]) ,.B(U[9]) ,.S(B[3]) ,.Out(V[9]));
    Mux Mux_inst05(.A(U[2]) ,.B(U[10]) ,.S(B[3]) ,.Out(V[10]));
```

```
    //B2
    Mux Mux_inst06(.A(1'b0) ,.B(V[0]) ,.S(B[2]) ,.Out(W[0]));
    Mux Mux_inst07(.A(1'b0) ,.B(V[1]) ,.S(B[2]) ,.Out(W[1]));
    Mux Mux_inst08(.A(1'b0) ,.B(V[2]) ,.S(B[2]) ,.Out(W[2]));
    Mux Mux_inst09(.A(1'b0) ,.B(V[3]) ,.S(B[2]) ,.Out(W[3]));
    Mux Mux_inst0A(.A(V[0]) ,.B(V[4]) ,.S(B[2]) ,.Out(W[4]));
    Mux Mux_inst0B(.A(V[1]) ,.B(V[5]) ,.S(B[2]) ,.Out(W[5]));
    Mux Mux_inst0C(.A(V[2]) ,.B(V[6]) ,.S(B[2]) ,.Out(W[6]));
    Mux Mux_inst0D(.A(V[3]) ,.B(V[7]) ,.S(B[2]) ,.Out(W[7]));
    Mux Mux_inst0E(.A(V[4]) ,.B(V[8]) ,.S(B[2]) ,.Out(W[8]));
    Mux Mux_inst0F(.A(V[5]) ,.B(V[9]) ,.S(B[2]) ,.Out(W[9]));
    Mux Mux_inst10(.A(V[6]) ,.B(V[10]) ,.S(B[2]) ,.Out(W[10]));

    //B1
    Mux Mux_inst11(.A(1'b0) ,.B(W[0]) ,.S(B[1]) ,.Out(X[0]));
    Mux Mux_inst12(.A(1'b0) ,.B(W[1]) ,.S(B[1]) ,.Out(X[1]));
    Mux Mux_inst13(.A(W[0]) ,.B(W[2]) ,.S(B[1]) ,.Out(X[2]));
    Mux Mux_inst14(.A(W[1]) ,.B(W[3]) ,.S(B[1]) ,.Out(X[3]));
    Mux Mux_inst15(.A(W[2]) ,.B(W[4]) ,.S(B[1]) ,.Out(X[4]));
    Mux Mux_inst16(.A(W[3]) ,.B(W[5]) ,.S(B[1]) ,.Out(X[5]));
    Mux Mux_inst17(.A(W[4]) ,.B(W[6]) ,.S(B[1]) ,.Out(X[6]));
    Mux Mux_inst18(.A(W[5]) ,.B(W[7]) ,.S(B[1]) ,.Out(X[7]));
    Mux Mux_inst19(.A(W[6]) ,.B(W[8]) ,.S(B[1]) ,.Out(X[8]));
    Mux Mux_inst1A(.A(W[7]) ,.B(W[9]) ,.S(B[1]) ,.Out(X[9]));
    Mux Mux_inst1B(.A(W[8]) ,.B(W[10]) ,.S(B[1]) ,.Out(X[10]));

    //B0
    Mux Mux_inst1C(.A(1'b0) ,.B(X[0]) ,.S(B[0]) ,.Out(T[0]));
    Mux Mux_inst1D(.A(X[0]) ,.B(X[1]) ,.S(B[0]) ,.Out(T[1]));
    Mux Mux_inst1E(.A(X[1]) ,.B(X[2]) ,.S(B[0]) ,.Out(T[2]));
    Mux Mux_inst1F(.A(X[2]) ,.B(X[3]) ,.S(B[0]) ,.Out(T[3]));
    Mux Mux_inst100(.A(X[3]) ,.B(X[4]) ,.S(B[0]) ,.Out(T[4]));
    Mux Mux_inst101(.A(X[4]) ,.B(X[5]) ,.S(B[0]) ,.Out(T[5]));
    Mux Mux_inst102(.A(X[5]) ,.B(X[6]) ,.S(B[0]) ,.Out(T[6]));
    Mux Mux_inst103(.A(X[6]) ,.B(X[7]) ,.S(B[0]) ,.Out(T[7]));
    Mux Mux_inst104(.A(X[7]) ,.B(X[8]) ,.S(B[0]) ,.Out(T[8]));
    Mux Mux_inst105(.A(X[8]) ,.B(X[9]) ,.S(B[0]) ,.Out(T[9]));
    Mux Mux_inst106(.A(X[9]) ,.B(X[10]) ,.S(B[0]) ,.Out(T[10]));

    assign Y = T[9:0];

endmodule
```

*Right Barrel Shifter,5bit Mux, 10bit Mux and Modular Exponent Subtractor are the same as the one's used in HFPFAdder with the code presented above

```
//Subtractor
module Subtractor
(
    input [10:0] A, B,
    input cin,
    output cout,
    output [10:0] R
);

    wire [11:0] C;
    assign C[0] = cin;
    assign cout = C[11];
    FullAdder FullAdder_inst0(.ai(A[0]) ,.bi(B[0]^cin) ,.cini(C[0]) ,.si(R[0]) ,.couti(C[1]));
    FullAdder FullAdder_inst1(.ai(A[1]) ,.bi(B[1]^cin) ,.cini(C[1]) ,.si(R[1]) ,.couti(C[2]));
    FullAdder FullAdder_inst2(.ai(A[2]) ,.bi(B[2]^cin) ,.cini(C[2]) ,.si(R[2]) ,.couti(C[3]));
    FullAdder FullAdder_inst3(.ai(A[3]) ,.bi(B[3]^cin) ,.cini(C[3]) ,.si(R[3]) ,.couti(C[4]));
    FullAdder FullAdder_inst4(.ai(A[4]) ,.bi(B[4]^cin) ,.cini(C[4]) ,.si(R[4]) ,.couti(C[5]));
    FullAdder FullAdder_inst5(.ai(A[5]) ,.bi(B[5]^cin) ,.cini(C[5]) ,.si(R[5]) ,.couti(C[6]));
    FullAdder FullAdder_inst6(.ai(A[6]) ,.bi(B[6]^cin) ,.cini(C[6]) ,.si(R[6]) ,.couti(C[7]));
    FullAdder FullAdder_inst7(.ai(A[7]) ,.bi(B[7]^cin) ,.cini(C[7]) ,.si(R[7]) ,.couti(C[8]));
    FullAdder FullAdder_inst8(.ai(A[8]) ,.bi(B[8]^cin) ,.cini(C[8]) ,.si(R[8]) ,.couti(C[9]));
    FullAdder FullAdder_inst9(.ai(A[9]) ,.bi(B[9]^cin) ,.cini(C[9]) ,.si(R[9]) ,.couti(C[10]));
    FullAdder FullAdder_instA(.ai(A[10]) ,.bi(B[10]^cin) ,.cini(C[10]) ,.si(R[10]) ,.couti(C[11]));

endmodule
```

```
1  // Multi 7 Segment display
2  module Multi7segdisplay
3  (
4      input [3:0] HEX0, HEX1, HEX2, HEX3,
5      input CLK, Reset, Load,
6      output logic [0:6] SEG,
7      output logic [3:0] CAT
8  );
9
10     logic [3:0] D0,D1,D2,D3;
11     logic [3:0]Y;
12     logic [1:0] SEL;
13
14
15     Controller Controller_inst
16     (
17        .MuxRate(CLK) ,   // input  MuxRate_sig
18        .Reset(Reset) ,   // input  Reset_sig
19        .SEL(SEL) , // output [1:0] SEL_sig
20        .CAT(CAT)   // output [3:0] CAT_sig
21     );
22
23     PIPO HEX0_inst
24     (
25        .D(HEX0) ,  // input [3:0] D_sig
26        .CLK(Load) ,   // input  CLK_sig
27        .CLR(Reset) ,  // input  CLR_sig
28        .Q(D0)   // output [3:0] Q_sig
29     );
30
31     PIPO HEX1_inst
32     (
33        .D(HEX1) ,  // input [3:0] D_sig
34        .CLK(Load) ,   // input  CLK_sig
35        .CLR(Reset) ,  // input  CLR_sig
36        .Q(D1)   // output [3:0] Q_sig
37     );
38
39     PIPO HEX2_inst
40     (
41        .D(HEX2) ,  // input [3:0] D_sig
42        .CLK(Load) ,   // input  CLK_sig
43        .CLR(Reset) ,  // input  CLR_sig
44        .Q(D2)   // output [3:0] Q_sig
45     );
46
47     PIPO HEX3_inst
48     (
49        .D(HEX3) ,  // input [3:0] D_sig
50        .CLK(Load) ,   // input  CLK_sig
51        .CLR(Reset) ,  // input  CLR_sig
52        .Q(D3)   // output [3:0] Q_sig
53     );
```

```
55      four2one four2one_inst
56      (
57          .SEL(SEL) , // input [1:0] SEL_sig
58          .D0(D0) ,   // input [3:0] D0_sig
59          .D1(D1) ,   // input [3:0] D1_sig
60          .D2(D2) ,   // input [3:0] D2_sig
61          .D3(D3) ,   // input [3:0] D3_sig
62          .Y(Y)    // output [3:0] Y_sig
63      );
64
65      HEX2Seven HEX2Seven_inst
66      (
67          .value(Y) , // input [3:0] value_sig
68          .seg(SEG)   // output [0:6] seg_sig
69      );
70
71  endmodule
```

```
1   //Controller FSM
2   module Controller
3   (
4       input MuxRate, Reset,
5       output reg [1:0] SEL,
6       output reg [3:0] CAT
7   );
8       reg [1:0] state, nextstate;
9
10      always @ (negedge MuxRate, negedge Reset)
11          if (Reset == 0) state <= 2'b0; else state <= nextstate;
12
13      always @ (state)
14          case({state})
15              2'b00:begin nextstate = 2'b01; SEL = 2'b00; CAT = 4'b1000; end
16              2'b01:begin nextstate = 2'b10; SEL = 2'b01; CAT = 4'b0100; end
17              2'b10:begin nextstate = 2'b11; SEL = 2'b10; CAT = 4'b0010; end
18              2'b11:begin nextstate = 2'b00; SEL = 2'b11; CAT = 4'b0001; end
19          endcase
20  endmodule
```

```
1   //PIPO register
2   module PIPO
3   (
4       input [3:0] D,
5       input CLK,CLR,
6       output logic [3:0] Q
7   );
8       always_ff @ (posedge CLK, negedge CLR)
9           begin
10              if(CLR==1'b0) Q <= 0;
11                  else if(CLK==1'b1) Q <= D;
12          end
13  endmodule
```

```
//Hex to Seven segment decoder
module HEX2Seven (
    input [3:0] value,
    output logic [0:6] seg);
    always_comb
        case(value)
            4'b0000: seg = 7'b1111110; //0
            4'b0001: seg = 7'b0110000; //1
            4'b0010: seg = 7'b1101101; //2
            4'b0011: seg = 7'b1111001; //3
            4'b0100: seg = 7'b0110011; //4
            4'b0101: seg = 7'b1011011; //5
            4'b0110: seg = 7'b1011111; //6
            4'b0111: seg = 7'b1110000; //7
            4'b1000: seg = 7'b1111111; //8
            4'b1001: seg = 7'b1110011; //9
            4'b1010: seg = 7'b1110111; //A
            4'b1011: seg = 7'b0011111; //b
            4'b1100: seg = 7'b1001110; //C
            4'b1101: seg = 7'b0111101; //d
            4'b1110: seg = 7'b1001111; //E
            4'b1111: seg = 7'b1000111; //F
            default: seg = 7'b0000001; //- for Error
        endcase
endmodule
```

```
// Four to one mux
module four2one
(
    input [1:0] SEL,
    input [3:0] D0,D1,D2,D3,
    output logic [3:0] Y
);
    always_comb
        case(SEL)
            2'b00: Y = D0;
            2'b01: Y = D1;
            2'b10: Y = D2;
            2'b11: Y = D3;
        endcase
endmodule
```

```
//Keypad Input module
module keypad_input // Depends: keypad_base(clock_div,keypad_fsm, keypad_decoder), shift_reg
#( parameter DIGITS = 4)
(  input clk,
    input reset,
    input [3:0] row,
    output [3:0] col,
    output [(DIGITS*4)-1:0] out,
    // Debug
    output [3:0] value,
    output trig
);
    keypad_base keypad_base(
        .clk(clk),
        .row(row),
        .col(col),
        .value(value),
        .valid(trig)
    );
    shift_reg #(.COUNT(DIGITS)) shift_reg(
        .trig(trig),
        .in(value),
        .out(out),
        .reset(reset)
    );
endmodule
```

```
//KeyPad Base Module
module keypad_base( // Depends: clock_div, keypad_fsm, keypad_decoder
    input clk,
    input [3:0] row,
    output [3:0] col,
    output [3:0] value,
    output valid,
    // Debug
    output slow_clock,
    output sense,
    output valid_digit,
    output [3:0] inv_row
);
    assign inv_row = ~row;
    clock_div #(.DIV(100000)) keypad_clock_divider(
        .clk(clk),
        .clk_out(slow_clock)
    );
    keypad_fsm keypad_fsm(
        .clk(slow_clock),
        .row(inv_row),
        .col(col),
        .sense(sense)
    );
    keypad_decoder #(.BASE(16)) keypad_key_decoder(
        .row(inv_row),
        .col(col),
        .value(value),
        .valid(valid_digit)
    );
    assign valid = valid_digit && sense;
endmodule
```

```
//Clock Divider Module
module clock_div
#(
parameter WIDTH = 32,
parameter DIV = 50 // Defaults to 1MHz
)
(
    input clk, reset,
    output clk_out
);
reg [WIDTH-1:0] r_reg;
wire [WIDTH-1:0] r_nxt;
reg clk_track;
    always @(posedge clk or posedge reset)
    begin
    if (reset)
        begin
            r_reg <= 0;
                clk_track <= 1'b0;
        end
    else if (r_nxt == DIV)
        begin
            r_reg <= 0;
            clk_track <= ~clk_track;
        end
    else
        r_reg <= r_nxt;
    end
    assign r_nxt = r_reg+1;
    assign clk_out = clk_track;
endmodule
```

```
//Keypad FSM module
module keypad_fsm(
    input clk,
    input [3:0] row,
    output reg [3:0] col,
    output sense,
// Debug
    output reg [3:0] state,
    output trig
);
    assign trig = row[0] || row[1] || row[2] || row[3];
    assign sense = state == 10;
    always @ (posedge clk)
        begin
            case (state)
                0: begin col = 4'b1111; state = 1; end
                1: if (trig) begin state = 2; end
                2: begin col = 4'b0001; state = 3; end
                3: if (trig) begin state = 10; end else begin state = 4; end
                4: begin col = 4'b0010; state = 5; end
                5: if (trig) begin state = 10; end else begin state = 6; end
                6: begin col = 4'b0100; state = 7; end
                7: if (trig) begin state = 10; end else begin state = 8; end
                8: begin col = 4'b1000; state = 9; end
                9: if (trig) begin state = 10; end else begin state = 0; end
                10: begin state = 11; end
                11: if (~trig) begin state = 0; end
            endcase
        end
endmodule
```

```
//Keypad Decoder module
module keypad_decoder
#(
    parameter BASE = 16 // Default Base 10 (Options: 10,16)
)
(
    input [3:0] row,
    input [3:0] col,
    output reg [3:0] value,
    output reg valid // Optional
);
    always @ (row, col)
    begin
        if (BASE == 10)
        begin
            // 1 2 3 A
            // 4 5 6 B
            // 7 8 9 C
            // E 0 F D
            case ({row, col})
                8'b0001_0001: begin value = 1; valid = 1; end // 1
                8'b0001_0010: begin value = 2; valid = 1; end // 2
                8'b0001_0100: begin value = 3; valid = 1; end // 3
                8'b0001_1000: begin value = 10; valid = 1; end // A
                8'b0010_0001: begin value = 4; valid = 1; end // 4
                8'b0010_0010: begin value = 5; valid = 1; end // 5
                8'b0010_0100: begin value = 6; valid = 1; end // 6
                8'b0010_1000: begin value = 11; valid = 1; end // B
                8'b0100_0001: begin value = 7; valid = 1; end // 7
                8'b0100_0010: begin value = 8; valid = 1; end // 8
                8'b0100_0100: begin value = 9; valid = 1; end // 9
                8'b0100_1000: begin value = 12; valid = 1; end // C
                8'b1000_0001: begin value = 14; valid = 1; end // E
                8'b1000_0010: begin value = 0; valid = 1; end // 0
                8'b1000_0100: begin value = 15; valid = 1; end // F
                8'b1000_1000: begin value = 13; valid = 1; end // D
                default: begin value = 0; valid = 0; end // Misc
            endcase
        end
```

```
            default: begin value = 0; valid = 0; end // Misc
         endcase
      end
      else if (BASE == 16)
      begin
         // 0 1 2 3
         // 4 5 6 7
         // 8 9 A B
         // C D E F
         case ({row, col})
            8'b0001_0001: begin value = 0; valid = 1; end // 0
            8'b0001_0010: begin value = 1; valid = 1; end // 1
            8'b0001_0100: begin value = 2; valid = 1; end // 2
            8'b0001_1000: begin value = 3; valid = 1; end // 3
            8'b0010_0001: begin value = 4; valid = 1; end // 4
            8'b0010_0010: begin value = 5; valid = 1; end // 5
            8'b0010_0100: begin value = 6; valid = 1; end // 6
            8'b0010_1000: begin value = 7; valid = 1; end // 7
            8'b0100_0001: begin value = 8; valid = 1; end // 8
            8'b0100_0010: begin value = 9; valid = 1; end // 9
            8'b0100_0100: begin value = 10; valid = 1; end // A
            8'b0100_1000: begin value = 11; valid = 1; end // B
            8'b1000_0001: begin value = 12; valid = 1; end // C
            8'b1000_0010: begin value = 13; valid = 1; end // D
            8'b1000_0100: begin value = 14; valid = 1; end // E
            8'b1000_1000: begin value = 15; valid = 1; end // F
            default: begin value = 0; valid = 0; end // Misc
         endcase
      end
      else
      begin
         value = 0; valid = 0; // Invalid Base
      end
   end
endmodule
```

```
//Shift Register Module
module shift_reg
#(
parameter COUNT = 4,
parameter WIDTH = 4
)
(
    input trig, reset, dir, // Left = 0, Right = 1
    input [WIDTH-1:0] in,
    output reg [(COUNT*WIDTH)-1:0] out
);
    always @ (posedge trig, negedge reset)
    begin
       if (~reset)
          out <= 0;
       else
       begin
          if (dir)
          begin // 1 = Right
             out <= out >> WIDTH;
             out[(COUNT*WIDTH)- 1:((COUNT*WIDTH)-1)-WIDTH] <= in;
          end
          else
          begin // 0 = Left
             out <= out << WIDTH;
             out[WIDTH-1:0] <= in;
          end
       end
    end
endmodule
```